jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel) Anaconda Toolbox

Patient Health Records – Data Preprocessing

Data Analyst Project | Missing Values + Outlier Handling

[ ]:

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from scipy import stats
from scipy.stats.mstats import winsorize

print("Libraries loaded")
```

```
Libraries loaded
```

Step 1 – Generate Dataset (550 rows)

[2]:
```
np.random.seed(42)
n = 550

patient_id = [f'P{str(i).zfill(4)}' for i in range(1, n+1)]

# Age - integer, some missing
age = np.random.randint(20, 81, n).astype(float)
age[np.random.choice(n, 30, replace=False)] = np.nan
```

jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel)

```
# Gender – Male/Female, some missing
gender = np.random.choice(['Male', 'Female'], n).astype(object)
gender[np.random.choice(n, 35, replace=False)] = np.nan

# Region – 4 categories, some missing
region = np.random.choice(['North', 'South', 'East', 'West'], n).astype(object)
region[np.random.choice(n, 40, replace=False)] = np.nan

# BMI – normal range with some outliers and missing
bmi = np.round(np.random.normal(26.5, 4.5, n), 2)
bmi[np.random.choice(n, 12, replace=False)] = np.random.choice([2.1, 3.5, 78.0, 85.0], 12, replace=True)
bmi[np.random.choice(n, 45, replace=False)] = np.nan

# Blood Pressure – some very high outlier values
bp = np.round(np.random.normal(118, 13, n), 1)
bp[np.random.choice(n, 18, replace=False)] = np.random.choice([210, 225, 240, 260], 18, replace=True)

# Cholesterol – outliers + missing
cholesterol = np.round(np.random.normal(195, 28, n), 1)
cholesterol[np.random.choice(n, 15, replace=False)] = np.random.choice([8, 12, 510, 560], 15, replace=True)
cholesterol[np.random.choice(n, 40, replace=False)] = np.nan

# Glucose – high spikes + missing
glucose = np.round(np.random.normal(95, 17, n), 1)
glucose[np.random.choice(n, 18, replace=False)] = np.random.choice([420, 480, 530, 590], 18, replace=True)
glucose[np.random.choice(n, 38, replace=False)] = np.nan

# Disease risk – binary target, no missing
disease_risk = np.random.choice([0, 1], n, p=[0.60, 0.40])

df = pd.DataFrame({
    'patient_id':     patient_id,
    'age':            age,
    'gender':         gender,
    'region':         region,
    'bmi':            bmi,
    'blood_pressure': bp,
    'cholesterol':    cholesterol,
```

jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3

```
    'blood_pressure': bp,
    'cholesterol':    cholesterol,
    'glucose':        glucose,
    'disease_risk':   disease_risk
})

print(f"Dataset shape: {df.shape}")
df.head(10)
```

```
Dataset shape: (550, 9)
```

[2]:

	patient_id	age	gender	region	bmi	blood_pressure	cholesterol	glucose	disease_risk
0	P0001	58.0	Female	East	30.04	95.9	179.2	71.3	0
1	P0002	71.0	Female	West	22.49	112.0	206.8	83.0	1
2	P0003	48.0	Female	North	21.72	110.0	204.3	90.5	1
3	P0004	34.0	Male	West	27.12	131.7	199.0	67.9	1
4	P0005	62.0	Female	South	25.34	104.8	163.2	NaN	1
5	P0006	27.0	Male	North	30.67	108.1	NaN	106.1	0
6	P0007	80.0	Female	South	28.69	113.0	257.2	119.3	0
7	P0008	40.0	Male	West	29.00	110.5	195.2	87.9	0
8	P0009	58.0	Female	West	NaN	135.8	165.7	107.9	0
9	P0010	77.0	NaN	South	NaN	103.2	NaN	62.5	1

[3]:

```
print(df.info())
print()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550 entries, 0 to 549
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
```

jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel)

```
---  ------          --------------  -----
 0   patient_id      550 non-null    object
 1   age             520 non-null    float64
 2   gender          515 non-null    object
 3   region          510 non-null    object
 4   bmi             505 non-null    float64
 5   blood_pressure  550 non-null    float64
 6   cholesterol     510 non-null    float64
 7   glucose         512 non-null    float64
 8   disease_risk    550 non-null    int64
dtypes: float64(5), int64(1), object(3)
memory usage: 38.8+ KB
None
```

[3]:

	age	bmi	blood_pressure	cholesterol	glucose	disease_risk
count	520.000000	505.000000	550.000000	510.000000	512.000000	550.000000
mean	50.842308	26.918040	121.413273	196.762745	107.017187	0.385455
std	17.623283	7.106704	24.978160	50.359874	69.560499	0.487146
min	20.000000	2.100000	74.800000	8.000000	50.400000	0.000000
25%	37.000000	23.540000	109.200000	176.700000	83.800000	0.000000
50%	51.000000	26.850000	117.500000	192.400000	95.100000	0.000000
75%	66.000000	30.040000	127.450000	213.900000	107.400000	1.000000
max	80.000000	85.000000	260.000000	560.000000	590.000000	1.000000

jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipyl

Part A – Handling Missing Values

[4]:
```
missing = df.isnull().sum()
pct = (missing / len(df) * 100).round(2)

report = pd.DataFrame({'Missing Count': missing, 'Missing %': pct})
report = report[report['Missing Count'] > 0].sort_values('Missing %', ascending=False)
print(report)

# bar chart
plt.figure(figsize=(8, 4))
plt.bar(report.index, report['Missing %'], color='salmon', edgecolor='black')
plt.title('Missing Values (%) per Column')
plt.ylabel('Missing %')
plt.xticks(rotation=30)
for i, v in enumerate(report['Missing %']):
    plt.text(i, v + 0.2, f'{v}%', ha='center', fontsize=9)
plt.tight_layout()
plt.show()
```

```
             Missing Count  Missing %
bmi                     45       8.18
region                  40       7.27
cholesterol             40       7.27
glucose                 38       6.91
gender                  35       6.36
age                     30       5.45
```

jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel)

Task 2a – Simple Imputer (Numerical) – BMI with Mean and Median

[5]:
```
df_a = df.copy()

mean_imp   = SimpleImputer(strategy='mean')
median_imp = SimpleImputer(strategy='median')

df_a['bmi_mean']   = mean_imp.fit_transform(df_a[['bmi']])
df_a['bmi_median'] = median_imp.fit_transform(df_a[['bmi']])

# show rows that were missing
was_missing = df['bmi'].isnull()
print("Rows where BMI was missing (first 10):")
print(df_a[was_missing][['patient_id', 'bmi', 'bmi_mean', 'bmi_median']].head(10))

# distribution comparison
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

axes[0].hist(df['bmi'].dropna(), bins=30, color='skyblue', edgecolor='black')
axes[0].set_title('Original BMI (with NaN)')

axes[1].hist(df_a['bmi_mean'], bins=30, color='lightgreen', edgecolor='black')
axes[1].set_title('BMI – Mean Imputed')

axes[2].hist(df_a['bmi_median'], bins=30, color='lightsalmon', edgecolor='black')
axes[2].set_title('BMI – Median Imputed')

plt.suptitle('Simple Numerical Imputer – BMI')
plt.tight_layout()
plt.show()
```

```
Rows where BMI was missing (first 10):
    patient_id  bmi  bmi_mean  bmi_median
8        P0009  NaN  26.91804       26.85
9        P0010  NaN  26.91804       26.85
23       P0024  NaN  26.91804       26.85
33       P0034  NaN  26.91804       26.85
52       P0053  NaN  26.91804       26.85
```

jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Trusted

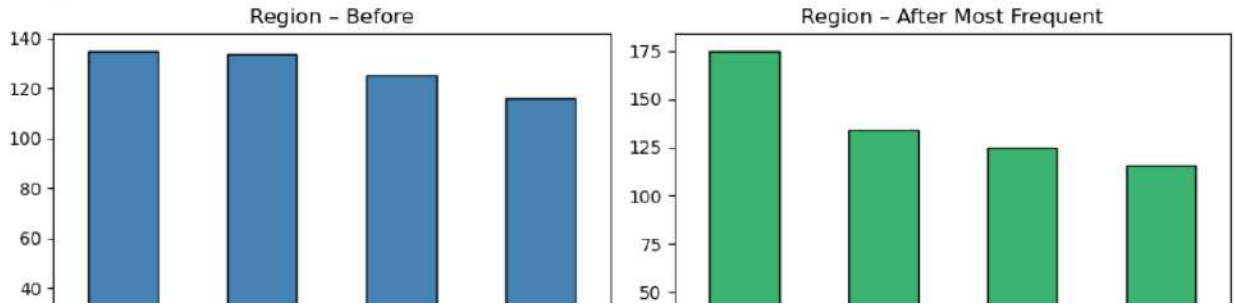
Code

JupyterLab Python 3 (ipykernel) Anaconda Toolbox

Task 2b – Simple Imputer (Categorical) – Region with Most Frequent

[6]:
```
df_b = df.copy()

cat_imp = SimpleImputer(strategy='most_frequent')
df_b['region'] = cat_imp.fit_transform(df_b[['region']]).ravel()

print(f"Most frequent region used: {df['region'].mode()[0]}")
print(f"Missing before: {df['region'].isnull().sum()}")
print(f"Missing after:  {df_b['region'].isnull().sum()}")

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
df['region'].value_counts().plot(kind='bar', ax=axes[0], color='steelblue', edgecolor='black')
axes[0].set_title('Region – Before')

df_b['region'].value_counts().plot(kind='bar', ax=axes[1], color='mediumseagreen', edgecolor='black')
axes[1].set_title('Region – After Most Frequent')

plt.tight_layout()
plt.show()
```

```
Most frequent region used: North
Missing before: 40
Missing after:  0
```



jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel) Anaconda Toolb

Task 2c – Most Frequent Imputation – Gender

[7]:
```
df_c = df.copy()

g_imp = SimpleImputer(strategy='most_frequent')
df_c['gender'] = g_imp.fit_transform(df_c[['gender']]).ravel()

print(f"Most frequent gender: {df['gender'].mode()[0]}")
print(f"Missing before: {df['gender'].isnull().sum()}")
print(f"Missing after:  {df_c['gender'].isnull().sum()}")

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
df['gender'].value_counts().plot(kind='pie', ax=axes[0], autopct='%1.1f%%',
                                 colors=['#66b3ff', '#ff9999', '#cccccc'])
axes[0].set_title('Gender – Before')
axes[0].set_ylabel('')

df_c['gender'].value_counts().plot(kind='pie', ax=axes[1], autopct='%1.1f%%',
                                   colors=['#66b3ff', '#ff9999'])
axes[1].set_title('Gender – After')
axes[1].set_ylabel('')

plt.tight_layout()
plt.show()
```

```
Most frequent gender: Male
Missing before: 35
Missing after:  0
```

jupyter Data_Cleanser Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel)

Task 2d – Missing Indicator + Random Sample Imputation

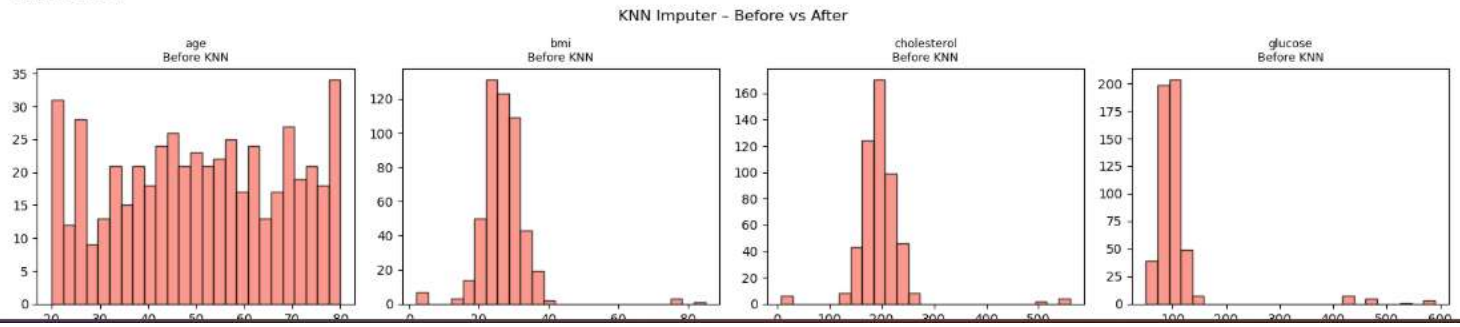
[8]:
```
df_d = df.copy()

num_cols = ['age', 'bmi', 'cholesterol', 'glucose']

for col in num_cols:
    # create flag column (1 = was missing)
    df_d[col + '_flag'] = df_d[col].isnull().astype(int)

    # fill missing with random sample from existing values
    null_mask = df_d[col].isnull()
    existing  = df_d[col].dropna().values
    df_d.loc[null_mask, col] = np.random.choice(existing, size=null_mask.sum(), replace=True)

print("Flag columns (how many were missing per column):")
flag_cols = [c for c in df_d.columns if c.endswith('_flag')]
print(df_d[flag_cols].sum())

print()
print("Missing after imputation:")
print(df_d[num_cols].isnull().sum())

print()
print(df_d[['patient_id', 'bmi', 'bmi_flag', 'glucose', 'glucose_flag']].head(12))
```

```
Flag columns (how many were missing per column):
age_flag            30
bmi_flag            45
cholesterol_flag    40
glucose_flag        38
dtype: int64

Missing after imputation:
age            0
bmi            0
cholesterol    0
glucose        0
dtype: int64
```

Task 2e – KNN Imputer

[9]:
```
df_e = df.copy()

num_cols = ['age', 'bmi', 'cholesterol', 'glucose']

knn = KNNImputer(n_neighbors=5)
df_e[num_cols] = knn.fit_transform(df_e[num_cols])

print("Missing after KNN:")
print(df_e[num_cols].isnull().sum())

fig, axes = plt.subplots(2, 4, figsize=(16, 7))
for i, col in enumerate(num_cols):
    axes[0, i].hist(df[col].dropna(), bins=25, color='salmon', edgecolor='black', alpha=0.8)
    axes[0, i].set_title(f'{col}\nBefore KNN', fontsize=9)

    axes[1, i].hist(df_e[col], bins=25, color='lightblue', edgecolor='black', alpha=0.8)
    axes[1, i].set_title(f'{col}\nAfter KNN', fontsize=9)

plt.suptitle('KNN Imputer - Before vs After')
plt.tight_layout()
plt.show()
```

```
Missing after KNN:
age            0
bmi            0
cholesterol    0
glucose        0
dtype: int64
```



File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel)

Task 2f – MICE Algorithm (Iterative Imputer)

[10]:
```
df_f = df.copy()

num_cols = ['age', 'bmi', 'cholesterol', 'glucose']

mice = IterativeImputer(max_iter=10, random_state=42)
df_f[num_cols] = mice.fit_transform(df_f[num_cols])

print("Missing after MICE:")
print(df_f[num_cols].isnull().sum())

print()
print(df_f[num_cols].describe().round(2))

fig, axes = plt.subplots(2, 4, figsize=(16, 7))
for i, col in enumerate(num_cols):
    axes[0, i].hist(df[col].dropna(), bins=25, color='plum', edgecolor='black', alpha=0.8)
    axes[0, i].set_title(f'{col}\nBefore MICE', fontsize=9)

    axes[1, i].hist(df_f[col], bins=25, color='palegreen', edgecolor='black', alpha=0.8)
    axes[1, i].set_title(f'{col}\nAfter MICE', fontsize=9)

plt.suptitle('MICE Algorithm – Before vs After')
plt.tight_layout()
plt.show()
```

```
Missing after MICE:
age            0
bmi            0
cholesterol    0
glucose        0
dtype: int64

          age     bmi  cholesterol  glucose
count  550.00  550.00       550.00   550.00
mean    50.84   26.92       196.76   107.02
std     17.14    6.81        48.49    67.11
min     20.00    2.10         8.00    50.40
25%     38.00   23.89       178.42    85.05
50%     50.85   26.87       195.70    96.40
75%     65.00   29.54       211.65   107.02
max     80.00   85.00       560.00   590.00
```

Part B – Handling Outliers

First we prepare a clean base (no NaN) using MICE + most frequent, then apply all 4 outlier methods.

Base Dataset – No Missing Values

[12]:
```
base = df_f.copy()

# fill remaining categoricals
for col in ['gender', 'region']:
    base[col] = base[col].fillna(base[col].mode()[0])

print("Missing values in base:")
print(base.isnull().sum())
print(f"Shape: {base.shape}")
```

```
Missing values in base:
patient_id        0
age               0
gender            0
region            0
bmi               0
blood_pressure    0
cholesterol       0
glucose           0
disease_risk      0
dtype: int64
Shape: (550, 9)
```

Task 3a – Z-Score Method (Cholesterol & Glucose)

[13]:
```
df_z = base.copy()

z_cols = ['cholesterol', 'glucose']

for col in z_cols:
    z = np.abs(stats.zscore(df_z[col]))
    outliers = (z > 3)
    print(f"{col} - outliers found: {outliers.sum()}")
    print(f"  values: {sorted(df_z.loc[outliers, col].tolist())[:8]}")
    df_z = df_z[~outliers]
```

Task 3b – IQR Method (BMI & Blood Pressure)

[14]:
```
df_iqr = base.copy()

iqr_cols = ['bmi', 'blood_pressure']

for col in iqr_cols:
    Q1 = df_iqr[col].quantile(0.25)
    Q3 = df_iqr[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outliers = (df_iqr[col] < lower) | (df_iqr[col] > upper)
    print(f"{col}:")
    print(f"  Q1={Q1:.2f}, Q3={Q3:.2f}, IQR={IQR:.2f}")
    print(f"  Lower fence={lower:.2f}, Upper fence={upper:.2f}")
    print(f"  Outliers found: {outliers.sum()}")
    df_iqr = df_iqr[~outliers]

print(f"\nRows before: {len(base)}  |  After: {len(df_iqr)}")

fig, axes = plt.subplots(1, 2, figsize=(11, 5))
for ax, col in zip(axes, iqr_cols):
    ax.boxplot([base[col].dropna(), df_iqr[col].dropna()],
               labels=['Before IQR', 'After IQR'],
               patch_artist=True,
               boxprops=dict(facecolor='lightcyan'),
               medianprops=dict(color='red', linewidth=2))
    ax.set_title(f'IQR - {col}')
    ax.set_ylabel(col)

plt.tight_layout()
plt.show()
```

```
bmi:
  Q1=23.89, Q3=29.54, IQR=5.65
  Lower fence=15.41, Upper fence=38.02
  Outliers found: 17
blood_pressure:
  Q1=109.20, Q3=127.70, IQR=18.50
  Lower fence=81.45, Upper fence=155.45
  Outliers found: 19
```

Task 4 – Winsorization (cap extreme values instead of removing rows)

[16]:
```
df_wins = base.copy()

wins_cols = ['bmi', 'blood_pressure', 'cholesterol', 'glucose']

for col in wins_cols:
    before_min = df_wins[col].min()
    before_max = df_wins[col].max()

    df_wins[col] = winsorize(df_wins[col], limits=[0.02, 0.02]).data

    print(f"{col}:")
    print(f"  Before -> min={before_min:.2f}, max={before_max:.2f}")
    print(f"  After  -> min={df_wins[col].min():.2f}, max={df_wins[col].max():.2f}")
    print()

fig, axes = plt.subplots(2, 4, figsize=(16, 7))
for i, col in enumerate(wins_cols):
    axes[0, i].hist(base[col], bins=28, color='orchid', edgecolor='black', alpha=0.8)
    axes[0, i].set_title(f'{col}\nBefore', fontsize=9)

    axes[1, i].hist(df_wins[col], bins=28, color='lightyellow', edgecolor='black', alpha=0.8)
    axes[1, i].set_title(f'{col}\nAfter Winsorize', fontsize=9)

plt.suptitle('Winsorization – Before vs After (2% each tail)')
plt.tight_layout()
plt.show()
```

```
bmi:
  Before -> min=2.10, max=85.00
  After  -> min=15.77, max=36.59

blood_pressure:
  Before -> min=74.80, max=260.00
  After  -> min=92.40, max=225.00

cholesterol:
  Before -> min=8.00, max=560.00
  After  -> min=132.50, max=259.30

glucose:
  Before -> min=50.40, max=590.00
  After  -> min=62.50, max=420.00
```

Part C – Final Clean Dataset

Task 6 – Build Final Clean Dataset

[18]:
```
# start from original raw data
df_final = df.copy()

# Step 1 - impute numericals with MICE
num_cols = ['age', 'bmi', 'cholesterol', 'glucose']
mice_final = IterativeImputer(max_iter=10, random_state=42)
df_final[num_cols] = mice_final.fit_transform(df_final[num_cols])

# Step 2 - impute categoricals with most frequent
for col in ['gender', 'region']:
    df_final[col] = df_final[col].fillna(df_final[col].mode()[0])

print("Missing after imputation:")
print(df_final.isnull().sum())

# Step 3 - handle outliers with winsorization (keeps all 550 rows)
treat_cols = ['bmi', 'blood_pressure', 'cholesterol', 'glucose']
for col in treat_cols:
    df_final[col] = winsorize(df_final[col], limits=[0.02, 0.02]).data
    df_final[col] = df_final[col].astype(float).round(2)

df_final['age'] = df_final['age'].round(0).astype(int)

print(f"\nFinal dataset shape: {df_final.shape}")
print()
df_final.head(10)
```

```
Missing after imputation:
patient_id        0
age               0
gender            0
region            0
bmi               0
blood_pressure    0
cholesterol       0
glucose           0
disease_risk      0
dtype: int64
```

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipyl

```
Final dataset shape: (550, 9)
```

[18]:

	patient_id	age	gender	region	bmi	blood_pressure	cholesterol	glucose	disease_risk
0	P0001	58	Female	East	30.04	95.9	179.20	71.30	0
1	P0002	71	Female	West	22.49	112.0	206.80	83.00	1
2	P0003	48	Female	North	21.72	110.0	204.30	90.50	1
3	P0004	34	Male	West	27.12	131.7	199.00	67.90	1
4	P0005	62	Female	South	25.34	104.8	163.20	107.02	1
5	P0006	27	Male	North	30.67	108.1	196.76	106.10	0
6	P0007	80	Female	South	28.69	113.0	257.20	119.30	0
7	P0008	40	Male	West	29.00	110.5	195.20	87.90	0
8	P0009	58	Female	West	26.84	135.8	165.70	107.90	0
9	P0010	77	Male	South	26.68	103.2	196.77	62.50	1

Final Dataset – Stats & Visuals

[19]:

```
print(df_final.describe(include='all').round(2))
```

```
       patient_id     age gender region     bmi  blood_pressure  cholesterol  \
count         550  550.00    550    550  550.00          550.00       550.00
unique        550     NaN      2      4     NaN             NaN          NaN
top         P0550     NaN   Male  North     NaN             NaN          NaN
freq            1     NaN    300    175     NaN             NaN          NaN
mean          NaN   50.85    NaN    NaN   26.75          121.06       195.04
std           NaN   17.14    NaN    NaN    4.49           22.58        26.82
min           NaN   20.00    NaN    NaN   15.77           92.40       132.50
25%           NaN   38.00    NaN    NaN   23.89          109.20       178.42
50%           NaN   51.00    NaN    NaN   26.87          117.50       195.70
75%           NaN   65.00    NaN    NaN   29.54          127.45       211.65
max           NaN   80.00    NaN    NaN   36.59          225.00       259.30

        glucose  disease_risk
count    550.00        550.00
unique      NaN           NaN
top         NaN           NaN
freq        NaN           NaN
```

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel)

disease_risk

Save Final Dataset to CSV

[21]:
```
df_final.to_csv('patient_health_records_clean.csv', index=False)
print("Saved: patient_health_records_clean.csv")
print(f"Shape: {df_final.shape}")
```

```
Saved: patient_health_records_clean.csv
Shape: (550, 9)
```

Task 7 – Brief Report

Which imputation method was most effective?

- **Simple Mean/Median** – Quick and easy but gets affected by outliers. Good starting point.
- **Most Frequent** – Works well for categorical columns like Gender and Region.
- **Random Sample** – Keeps the original distribution but adds some randomness.
- **KNN** – Better because it looks at similar patients to fill the gap, not just a single value.
- **MICE** – Best overall. It uses all columns together to estimate missing values. More accurate than the rest.

Winner: MICE – because it considers relationships between features while imputing.

Which outlier method preserved data quality best?

- **Z-Score** – Removes rows where values are 3 standard deviations away. Simple but removes real patients.
- **IQR** – Uses Q1/Q3 fences. Standard approach, removes rows too.
- **Percentile Capping** – No rows removed. Just clips values at 1st and 99th percentile. Safe choice.
- **Winsorization** – Similar to percentile capping. Caps top and bottom 2%. No data loss.

Winner: Winsorization – No rows are deleted. All 550 patients stay. Extreme values are just capped.

How did data cleaning improve the dataset?

- Reduced total missing values from ~228 to **0**